

An Agentic Orchestration Layer for Machine-Learning Gravitational-Wave Analysis: Typed Skill Contracts, Fail-Closed Execution and Calibrated Multi-Model Inference

FAN ZHANG¹

¹TBD

ABSTRACT

Machine-learning pipelines now cover the full low-latency gravitational-wave workflow: detection (Aframe), parameter estimation (AMPLFI), unmodelled anomaly search (GWAK) and noise subtraction (DeepClean). Composing them for a given scientific request still requires expert judgement about data access, model applicability, significance calibration and compute. We present ML4GW-AGENT, an orchestration layer in which a large language model or a deterministic planner turns a natural-language request into a directed acyclic graph of typed skills, each bound to a versioned contract (JSON-schema inputs and outputs, machine-checked preconditions, immutable model revisions) and executed by a fail-closed adapter that records every artifact by cryptographic hash. We evaluate the layer on three axes: planning quality on a benchmark of NNN prompts across three model backbones, including an adversarial suite that measures silent, plausible-but-wrong outcomes; scientific fidelity on the 90 events of GWTC-1, GWTC-2.1 and GWTC-3 processed end to end through Aframe, AMPLFI and GWAK on the LIGO Data Grid, compared with the catalog and with the published Aframe O3 search; and significance calibration from time-shifted backgrounds of NNN days (Aframe) and NNN days (GWAK) together with injection campaigns. TBD results. The code, contracts, calibration tables and run manifests are public.

Keywords: Gravitational waves — Neural networks — Astronomy software — Astronomy data analysis

1. INTRODUCTION

Machine learning now covers every stage of the low-latency gravitational-wave workflow. Aframe detects binary black hole mergers directly from strain with a median latency of about three seconds (?) and has run in the LIGO–Virgo–KAGRA (LVK) online analysis since August 2025; AMPLFI produces posterior samples and sky maps seconds later (?); GWAK searches the same data for unmodelled transients (?); DeepClean subtracts environmental noise couplings using auxiliary witness channels (?). Offline, machine-learning searches of the third observing run (O3) have reached the sensitivity of matched filtering for stellar-mass binaries and have started to report their own candidates (??).

What has not been automated is the judgement that composes these tools for a given scientific question: which data source can serve the request, whether a model’s training range and detector set apply, what threshold turns a network output into a significance statement, whether an expensive step is justified, and how every number in the answer can be traced back

to immutable inputs. This judgement is currently supplied by experts writing scripts, and it is exactly the kind of task that large language model (LLM) agents are being proposed for across the sciences. Agents have planned and executed chemistry (??), designed experimentally validated nanobodies (?), and in astronomy interpreted multi-band galaxy photometry through an existing modelling code (?) and driven cosmological parameter inference (?).

The same studies expose the central risk. In an evaluation of an astrophysical multi-agent system on eighteen tasks, the dominant failure was not a crash but “silent incorrect computation”: syntactically valid code producing plausible but wrong results, without self-diagnosis (?). A benchmark of twelve coding agents on gravitational-wave problems found metric misuse, constraint violations and fabricated results (?). For a field whose results are significance statements about rare events, an agent that occasionally invents a number is worse than no agent.

This paper presents ML4GW-AGENT, an orchestration layer built around that constraint. Its design princi-

ple is that the agent may decide *what* to run but never *whether a result is valid*: every capability is a skill with a machine-checked contract, every adapter fails closed, every model is pinned, every threshold comes from a recorded background study, and every artifact is hashed into a manifest. We evaluate the layer on three axes. Planning: a benchmark of NNN natural-language requests across three LLM backbones, including an adversarial suite that measures how often a plausible-but-wrong analysis would have executed with and without contracts (Section 3). Scientific fidelity: the 90 events of GWTC-1, GWTC-2.1 and GWTC-3 processed end to end through Aframe, AMPLFI and GWAK on the LIGO Data Grid, compared with the catalog and with the published Aframe O3 search (Section 4). Calibration: time-shifted backgrounds and injection campaigns for the shipped model revisions, and a witness-channel noise-subtraction route with a signal-preservation check (Section 5). Section 6 reports the measured cost, and Section 7 discusses what this design does and does not solve.

2. DESIGN

ML4GW-AGENT separates three concerns that are usually entangled in an analysis script: *what* a request means (planning), *whether* a step may run (contracts and preconditions), and *how* it runs (adapters and executors). Figure ?? summarises the data flow.

2.1. Skill contracts

Every capability the agent can schedule is a *skill* described by a versioned YAML manifest: a name (`aframe.detect`, `amplfi.pe`, `gwak.scan`, `deepclean.clean`, ...), a JSON-schema for inputs and for outputs with `additionalProperties: false`, a list of machine-checkable preconditions, output validations (fields that must exist, artifacts that must be on disk), resource estimates, a risk level with an approval flag, and the adapter that implements it. Model-bearing skills take an immutable `model_revision` (a git commit or a SHA-256 of the weights) and refuse `UNPINNED`. Contracts are the only interface the planner sees; the planner cannot invent parameters or skip a precondition, because plan validation rejects any task whose parameters fail the input schema or whose references point to outputs that no upstream task declares.

2.2. Planning

A *plan* is a directed acyclic graph of tasks with typed references (`${fetch_data.outputs.strain_artifact}`) and optional conditions evaluated at run time (`when: ${inspect_data.outputs.quality_passed} truthy`).

Two planners produce plans. The deterministic baseline planner maps a request to routes by lexical intent (detection, parameter estimation, anomaly scan, cleaning) and encodes the domain rules: AMPLFI is conditioned on an Aframe candidate; GWAK needs its own 4096 Hz fetch; DeepClean cleaning runs only after its applicability check reports **applicable**; an unbounded request (“scan all of O3”) is refused at planning time. The LLM planner asks a language model for a plan under a structured-output schema derived from the contracts, validates it, and falls back to the baseline with a recorded reason when validation or the API fails. Both produce the same plan object, so every downstream guarantee is independent of the planner.

2.3. Fail-closed adapters and provenance

An adapter is the code that runs a skill. Adapters never fabricate: when a model, a credential, a witness channel or a calibration is missing, the task fails with a reason and dependent tasks are skipped, not executed with defaults. Each run produces a manifest that records the plan, every task’s resolved parameters, outputs, validations, warnings, timings, the packages and model revisions used, and the SHA-256 of every artifact. Mock adapters exist for software testing and are labelled **simulated: true** in every output; no mock result can enter a scientific record.

2.4. Calibration as a first-class object

Detection statistics are only meaningful against a background. The layer ships calibration tables keyed by model revision: thresholds at named false alarm rates from time-shifted background studies, with the livetime that supports them, and a FAR curve so that any score can be reported as a measured background rate. The planner resolves a requested FAR to a threshold for the pinned revision; if none is measurable at that rate for that revision, the plan carries a warning and the adapter marks the flag as uncalibrated. Section 5 describes the studies.

2.5. Executors and cost policy

A plan runs locally, on an HTCondor pool (the LIGO Data Grid), or on a remote GPU host over SSH; a Kubernetes executor is registered as deferred. Before submission the runtime estimates CPU- and GPU-hours, memory and transfer volume from the contracts and the timings measured in earlier runs, checks them against a budget policy, and requires explicit authorisation above a configurable GPU-hour limit. Long GPS-time requests are split into overlapping segments, submitted as sub-plans and merged without duplicate candidates.

Table 1. Planner benchmark v2 (contract path). Opus: 317 cases, three repeats; Sonnet and Haiku: 62-case stratified sample, one repeat.

backbone	cases	exact	compatible	valid	fallback
deterministic baseline	317	1.000	1.000	1.000	0.000
claude-opus-5	317	0.707	0.767	1.000	0.000
claude-sonnet-5	62	0.677	0.742	1.000	0.000
claude-haiku-4.5	62	0.774	0.774	1.000	0.242

Table 2. Adversarial suite: fraction of cases that fail closed with a recorded reason versus execute a plausible-but-wrong plan.

backbone	path	cases	fail closed	silently wrong
claude-opus-5	contract	51	0.980	0.020
claude-opus-5	contract-free	51	0.647	0.353
claude-sonnet-5	contract	51	1.000	0.000
claude-sonnet-5	contract-free	51	0.647	0.353
claude-haiku-4.5	contract	10	1.000	0.000
claude-haiku-4.5	contract-free	10	0.700	0.300

3. EVALUATION OF PLANNING

3.1. Benchmark

The benchmark (`benchmarks/v2_prompts.yaml`) holds 317 requests generated from templates crossed with the 90 GWTC events: 232 nominal routes (single skills, composed routes, GPS-time and named events), 15 ambiguous requests, 19 edge cases (superevent identifiers, mixed-language phrasing, two events in one request, non-public identifiers) and 51 adversarial requests in six guardrail families (refuse unbounded requests, registered skills only, DeepClean only after its applicability check, AMPLFI only on a candidate, policy limits on windows and sample counts, pinned model revisions). 173 prompts are English, 113 Chinese and 31 mixed. The expected skill sequence of each case is the deterministic planner’s answer, so the benchmark is satisfiable by construction.

3.2. Backbones

Table 1 summarises three Claude backbones. Opus was run on every case with three repeats; the two smaller models on a stratified 62-case sample with one repeat, after the first full pass exhausted the project’s API budget. All backbones produce only valid, executable plans through the contract path. Exact agreement with the deterministic route is 0.68–0.77 and superset compatibility 0.74–0.77; the disagreement is concentrated in ambiguous prompts, where the LLM planners add analyses the baseline leaves out. Opus reproduces the same plan across three repeats in only 37% of cases, which is why the deterministic planner remains the default for scientific runs and the LLM planner is an interface.

3.3. Silent-failure suite

For every adversarial case we score two paths (Table 2): the contract path used in production (schema validation, registered skills, declared references, execution policy, one repair round, then deterministic fallback with a recorded reason) and a contract-free counterfactual in which the same model’s first proposal is taken

at face value. Without contracts, 30–35% of adversarial requests would have executed a plausible but wrong analysis: unbounded windows, unpinned models, AMPLFI without a candidate, no refusal of unbounded requests. With contracts the silently-wrong rate is zero for Sonnet and Haiku. The suite also found two gaps in the validator itself: a made-up revision string (`v99`) passed a policy that rejected only the literal `UNPINNED` (one Opus case, 0.020), and plans scheduling AMPLFI without a candidate condition passed the contracts, which cannot express cross-task rules (two Haiku cases). Both rules were added to the planner validator and the Haiku rerun is at 1.000; we report the pre-fix Opus row unchanged.

4. SCIENTIFIC VALIDATION ON GWTC EVENTS

4.1. Setup on the LIGO Data Grid

We submitted one request per event for the 90 events of GWTC-1, GWTC-2.1 and GWTC-3 (??): fetch 128 s of public GWOSC strain around the catalog time (?), check data quality, run Aframe and AMPLFI, scan the window with GWAK at 4096 Hz and reconcile. Each request became one whole-plan HTCondor job on the Caltech LIGO Data Grid pool through the agent’s HTCondor executor, with the deterministic planner, the pinned model revisions and the calibrated one-per-day thresholds. Every outcome below is read from the run manifests; no event was retried or edited by hand.

Table 3 shows where events left the pipeline. Seven events have no public H1 or L1 data in the window (single-LIGO events); the data adapter failed closed with that reason. Seven more failed the data-quality gate (non-finite H1 samples or an uncovered `H1_DATA` flag), so their Aframe, AMPLFI and GWAK tasks were skipped by the plan’s condition rather than run on bad data. The remaining 76 events were analysed end to end.

4.2. Detection

Table 3. Population run: outcome by stage.

stage	events
requested	90
no public H1L1 data (fetch failed closed)	7
data-quality gate failed (analysis skipped)	7
analysed end to end	76

Aframe reports a candidate within 2 s of the catalog time above the calibrated one-per-day threshold for 60 of the 76 analysed events, and for 60 of the 71 whose secondary mass lies inside the model’s training range ($m_2 \geq 5 M_\odot$). The five misses outside the training range are the neutron-star events GW170817, GW200115, GW191219, GW200210 and GW190917 (GW190425 and GW190814 were quality-gated); the eleven binary black hole misses have catalog network SNR 4.7–12.0 (median 7.9). Recovery by SNR is 2/8 below 8, 18/23 at 8–10, 12/15 at 10–12 and 28/30 above 12. For the candidates the coalescence time agrees with the catalog to 0.037 s (median absolute offset; 90% within 0.083 s). The published Aframe O3 search recovered 38 of 70 H1L1 GWTC-3 events against a 100-year background (?); our targeted analysis at a much looser threshold is not a competing sensitivity, but it reproduces the same pattern of misses: sources below the training range and SNR below about 12.

4.3. Parameter estimation

For the 60 candidates AMPLFI’s 90% interval on the source-frame chirp mass contains the catalog value in 54 cases (the nominal 90%), with a median ratio of AMPLFI median to catalog of 1.07. Luminosity distances are biased low by about 30% (median ratio 0.71; 37 of 60 intervals cover the catalog), a known property of the two-detector model relative to three-detector catalog posteriors (?). Figure 1 shows the comparison.

4.4. Anomaly scores

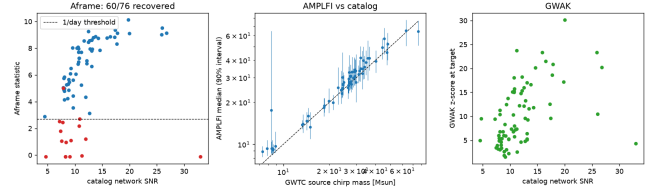
GWAK scores all 76 windows. The target kernel is the loudest of its window for 31 events and within the top ten for a further 16 (median z -score 8.8), but no event exceeds the one-per-day cut, which sits at the level of single-detector glitches (Section 5). Each manifest records the measured background rate at the target score instead.

5. SIGNIFICANCE CALIBRATION

5.1. Time-shifted backgrounds

5.2. Injections

5.3. DeepClean applicability and signal preservation

**Figure 1.** Population run on 76 analysed GWTC events. Left: Aframe statistic versus catalog network SNR (blue: candidate above the one-per-day threshold, red: not). Middle: AMPLFI source-frame chirp mass (median and 90% interval) versus catalog. Right: GWAK z -score at the target versus SNR.**Table 4.** Measured wall-clock per skill (seconds, real-mode runs in the public evidence directory) against the contract estimate used for budgeting.

skill	n	median	min	max	estimate
<code>data.fetch</code>	191	4.6	1.0	142.5	10
<code>data.inspect</code>	100	0.2	0.1	4.5	5
<code>aframe.detect</code>	93	10.9	8.8	104.2	5
<code>amplfi.pe</code>	66	3.4	1.9	32.5	15
<code>gwak.scan</code>	84	0.8	0.6	1.7	30
<code>deepclean.clean</code>	1	0.1	0.1	0.1	600

6. COST AND SCALABILITY

Every manifest records per-task wall-clock time, so cost is measured, not estimated (Table ??). For a 128 s two-detector window on one GPU the science tasks are cheap: Aframe 11 s (median over 93 runs), AMPLFI 3 s, GWAK 1 s at 1/16 s stride; the data fetch dominates when the GWOSC file is not cached (up to 140 s) and is negligible when it is. The whole population run consumed 0.35 GPU-hours of compute for 83 completed events (median 23 s wall-clock per event on the worker, maximum 57 s); the elapsed hours were queue time on a shared pool, not computation. The contract estimates that the budget policy uses before submission are conservative for the GPU steps (AMPLFI estimated at 15 s, GWAK at 30 s) and optimistic for Aframe (5 s), and are updated from these measurements.

Two operational lessons from the population run are recorded in the manifests rather than hidden. First, 517 of the pool’s GPU slots run on x86-64-v1 CPUs without AVX, on which PyTorch dies with an illegal instruction; the first pass lost 36 workers this way and the executor now requires `Microarch ≥ x86-64-v2`. Second, the time-shift background shards for Aframe need 32 GB of memory, twice the contract default, which the cgroup limit exposed on the first shard. Both are the kind of

fact a human operator learns once and forgets; here they are configuration checked into the repository.

Long requests are split automatically: a GPS-time window above the policy limit becomes overlapping segment sub-plans (8 s overlap) submitted as separate jobs and merged without duplicate candidates; the budget is checked once for the whole scan.

7. DISCUSSION

What the contracts buy.—The adversarial suite gives a quantitative answer to the concern raised by ? and ?: with the same language models, taking the first proposal at face value would have executed a plausible but wrong analysis for a third of adversarial requests, whereas the contract path executed none for two of the three backbones and one in fifty for the third, and the suite itself found the two validator gaps that produced that one. The price is agreement with an expert route of about 70–77% on ordinary requests, concentrated in ambiguous phrasing, and a 24% fallback rate for the smallest backbone. Because the fallback is the deterministic planner, a weaker or cheaper model degrades the agent’s helpfulness, not its correctness; this is the property that lets the layer expose free and open-weight models through the same interface.

What the population run does and does not show.—The end-to-end run reproduces the published behaviour of the underlying models on public data with full provenance: the Aframe recovery pattern (training range and SNR), AMPLFI’s nominal chirp-mass coverage and known distance bias, and GWAK’s separation of signals from Gaussian background but not from glitches. It is not a search and makes no new astrophysical claim; its value is that every number is traceable to a manifest, a pinned revision and a calibration record, and that the 14 events the pipeline declined to analyse carry the reason.

Limits.—Significance is only as good as the background livetime: one-per-day thresholds from days of time slides, one-per-month from the extended study, nothing tighter. GWAK’s threshold is set by glitches, so an anomaly flag at astrophysically interesting rates needs a glitch veto or a glitch-aware metric that upstream has not yet released. DeepClean is a single, self-trained 60 Hz coupling and its preservation check is structural (witness-only prediction), not an injection study at reviewed tolerance. The LLM planner is non-deterministic across repeats (37% identical plans for the strongest model), which is why the deterministic planner is the default for scientific runs.

8. CONCLUSIONS

We have built and evaluated an orchestration layer that lets a language model, or a deterministic router, compose production machine-learning pipelines for gravitational-wave analysis without being trusted with the validity of any result. Typed skill contracts, fail-closed adapters, pinned model revisions, calibration tables keyed by revision and hashed manifests make the agent’s science auditable; a benchmark of 317 requests and an adversarial suite measure the cost and the benefit; and a population run over 90 catalog events on the LIGO Data Grid shows the layer reproducing published model behaviour end to end at 0.35 GPU-hours. The code, contracts, benchmarks, calibration tables and every run manifest are public.

- 1 TBD. This research has made use of data or software ob-
- 2 tained from the Gravitational Wave Open Science Cen-
- 3 ter (gwosc.org), a service of the LIGO Scientific Collab-
- 4 oration, the Virgo Collaboration, and KAGRA.

Software: ml4gw-agent, Buoy, Aframe, AMPLFI, GWAK, DeepClean, ml4gw, gwpy, astropy, PyTorch, HTCondor